

 UNIVERSIDAD DE ANTIOQUIA 1803	PROGRAMA OFICIAL DE CURSO (Pregrado y Posgrado)
	UNIVERSIDAD DE ANTIOQUIA

1. INFORMACIÓN GENERAL			
Nombre del Curso:	ARQUITECTURA DE SOFTWARE		
Programa académico al que pertenece:	ING DE SISTEMAS VIRTUAL		
Unidad Académica:	Facultad de Ingeniería		
Vigencia:	2024-1 2024-2	- Código curso:	2554608
Tipo de curso:	Profesional		
CARACTERÍSTICAS DEL CURSO			
Habitable (H):	NO	Validable (V):	NO
Clasificable (C):	NO	Evaluación de suficiencia (Posgrado):	NO
Modalidad educativa del curso:	Virtual		
Área, núcleo o componente de la organización curricular a la que pertenece el curso	Básicas		
Número de créditos académicos:	3		
Horas totales de interacción estudiante-profesor:	64	Horas totales de trabajo independiente:	80
Horas totales del curso del semestre:	144		
Horas totales de actividades académicas teóricas:	0	Horas totales de actividades académicas prácticas:	0
Horas totales de actividades académicas teórico-prácticas:	64		

PROGRAMAS ACADÉMICOS EN LOS CUALES SE OFRECE EL CURSO	
506 - INGENIERÍA DE SISTEMAS - REGIÓN Versión: 5	
Pre-requisitos:	2554506 - ANÁLISIS Y DISEÑO DE SISTEMAS II
Co-requisitos:	2554609 - BASES DE DATOS Y LABORATORIO
552 - INGENIERÍA DE SISTEMAS VIRTUAL-SEDE MEDELLIN Versión: 1	
Pre-requisitos:	2554506 - ANÁLISIS Y DISEÑO DE SISTEMAS II
Co-requisitos:	2554609 - BASES DE DATOS Y LABORATORIO
506 - INGENIERÍA DE SISTEMAS - REGIÓN Versión: 4	
Pre-requisitos:	2554506 - ANÁLISIS Y DISEÑO DE SISTEMAS II
Co-requisitos:	2554609 - BASES DE DATOS Y LABORATORIO

2. RELACIONES CON EL PERFIL

El curso de arquitectura de software contribuye al cumplimiento de los siguientes perfiles profesionales definidos en el programa de Ingeniería de sistemas:

Desarrollo de software funcional, eficiente, robusto, fiable, usable, seguro, mantenible, y portable.

Desarrollo de infraestructuras tecnológicas y de comunicaciones, que respondan a las necesidades de las organizaciones.

La creación, planeación y dirección de proyectos informáticos y organizaciones de base tecnológica.

La habilidad de comunicación interpersonal para trabajar en equipos interdisciplinarios.

La capacidad de comunicarse de forma oral y escrita en español e inglés.

Liderar equipos de trabajo de manera proactiva para resolver problemas.

La toma de decisiones técnicas o administrativas de las organizaciones reconociendo y aplicando el marco regulatorio y estándares de calidad definidos para la profesión, asumiendo las responsabilidades éticas que la comunidad profesional y la sociedad le confieren y exigen.

De acuerdo a lo anterior el propósito del curso de Arquitectura de Software está diseñado para proporcionar a los estudiantes los conocimientos y habilidades necesarios para comprender, diseñar y evaluar arquitecturas de software efectivas y eficientes.

Este curso se alinea estrechamente con los perfiles del programa académico de Ingeniería de Sistemas y contribuye al desarrollo integral de los estudiantes en varios aspectos:

Entendimiento Profundo de la Arquitectura de Software:

El curso busca brindar a los estudiantes una comprensión sólida de los principios fundamentales de la arquitectura de software, incluyendo patrones, estilos arquitectónicos y buenas prácticas.

Aplicación Práctica en Proyectos de Ingeniería:

A través de proyectos y ejercicios prácticos, los estudiantes tienen la oportunidad de aplicar los conceptos aprendidos en situaciones del mundo real. Esto fortalece su capacidad para abordar desafíos arquitectónicos en el desarrollo de sistemas de software de cualquier tamaño.

Desarrollo de Habilidades de Toma de Decisiones:

El curso busca cultivar habilidades en la toma de decisiones relacionadas con la arquitectura de software. Los estudiantes aprenden a evaluar diferentes enfoques arquitectónicos y a seleccionar la mejor solución para satisfacer los requisitos específicos del sistema.

Colaboración y Comunicación Efectiva:

Los estudiantes aprenden a expresar claramente las decisiones arquitectónicas y a trabajar en equipo para lograr soluciones sólidas.

Adaptabilidad a Tecnologías Emergentes:

Dado el entorno tecnológico en constante evolución, el curso tiene como objetivo preparar a los estudiantes para adaptarse a nuevas tecnologías y tendencias en arquitectura de software.

Alineación con las Necesidades del Mercado Laboral: Prepara a los estudiantes para roles como arquitectos que requieren la capacidad de diseñar y liderar la implementación de arquitecturas de software robustas y escalables.

3. INTENCIONALIDADES FORMATIVAS

- ☒ Desarrollar componentes de software mediante la aplicación de buenas prácticas, métodos y estándares internacionales de ingeniería de software.
- ☒ Modelar aspectos dinámicos y estáticos empleando el lenguaje de modelado UML en el proceso de diseño de vistas arquitectónicas.
- ☒ Proponer arquitecturas de software completas para aplicaciones modernas y de calidad.
- ☒ Implementar estilos arquitectónicos modernos para diferentes dominios de aplicación
- ☒ Analizar los diferentes enfoques metodológicos y patrones arquitectónicos para abordar el proceso de desarrollo de arquitectura de software.
- ☒ Comprender el funcionamiento, características e integración de los diferentes tipos de servicios Web para la integración exitosa de componentes de software modernos.
- ☒ Trabajar en equipo eficazmente en el desarrollo de proyectos de ingeniería de software siendo ético, respetuoso, persistente y creativo.
- ☒ Analiza los principios de seguridad en la etapa de codificación del ciclo de vida de un software empleando el estudio de casos para establecer las prácticas de codificación y gestión de una arquitectura segura

4. APORTES DEL CURSO A LA FORMACIÓN INTEGRAL Y A LA FORMACIÓN EN INVESTIGACIÓN

El curso de Arquitectura de Software contribuye significativamente a la formación integral de los estudiantes, abordando las racionalidades ética, política, estética y lógica, así como fomentando la formación en investigación.

Aquí se detallan los aportes en cada una de estas dimensiones:

Racionalidad Ética:

El curso aborda la racionalidad ética al destacar la importancia de tomar decisiones éticas en el diseño y desarrollo de sistemas de software. Se enfoca en cuestiones relacionadas con la privacidad, la seguridad, el impacto en las empresas y la equidad en

el acceso a la tecnología. Los estudiantes son guiados para considerar las implicaciones éticas de las decisiones arquitectónicas en la creación de soluciones tecnológicas.

Racionalidad Política:

Al explorar diferentes enfoques y estilos arquitectónicos, el curso promueve la racionalidad política al capacitar a los estudiantes para comprender y evaluar las implicaciones políticas de las decisiones tecnológicas. Se fomenta la conciencia sobre cómo las decisiones arquitectónicas pueden afectar a diferentes stakeholders y cómo alinear el diseño de software con los objetivos organizativos y sociales de acuerdo con los drivers arquitectónicos.

Racionalidad Estética:

La racionalidad estética se aborda a través de la consideración de la elegancia y la simplicidad en el diseño arquitectónico. Los estudiantes aprenden a apreciar la estética y organización en la estructura y organización de los sistemas de software. Se fomenta la búsqueda de soluciones que no solo sean funcionales y eficientes, sino también estéticamente agradables y comprensibles.

Racionalidad Lógica:

Dada la naturaleza técnica del curso, la racionalidad lógica es central. Los estudiantes adquieren habilidades lógicas al comprender y aplicar principios de diseño arquitectónico coherentes y racionales. Se les desafía a desarrollar soluciones lógicas a problemas complejos, fortaleciendo su pensamiento analítico y capacidad de razonamiento lógico.

Formación en Investigación:

El curso fomenta la formación en investigación al introducir a los estudiantes en los fundamentos teóricos y prácticos de la arquitectura de software. Se les anima a explorar literatura académica, analizar tendencias emergentes y participar en la resolución de problemas prácticos. Los proyectos y actividades prácticas ofrecen oportunidades para aplicar métodos de investigación en la solución de problemas arquitectónicos específicos.

Intencionalidades Formativas:

Las intencionalidades formativas del curso se alinean con la preparación de profesionales de la ingeniería de sistemas capaces de abordar desafíos éticos, políticos, estéticos y lógicos en el diseño y desarrollo de software. Se busca que los estudiantes se conviertan en profesionales reflexivos, éticos y competentes en la toma de decisiones arquitectónicas

5. DESCRIPCIÓN DE LOS CONOCIMIENTOS Y/O SABERES

Unidad 1. Fundamentos en Arquitecturas de SW

Número de semanas: 5

- ☒ Introducción a los ambientes Backend, Frontend y Fullstack
- ☒ Definiciones de Arquitectura de Software
- ☒ Historia de la Arquitectura de Software
- ☒ Evolución de la Arquitectura de Software (Tecnologías y Frameworks)
- ☒ Diseño vs Arquitectura

- ☒ Integración de UML para generación de Vistas Arquitectónicas
- ☒ El rol del Arquitecto de Software (Roles, Tipos de Arquitectos, Cualidades del Arquitecto)
- ☒ Requisitos No Funcionales – Norma ISO 25010 (Como se cumplen desde el punto de vista de la Arquitectura)
- ☒ Conectores y Protocolos de Garlan y Shaw
- ☒ Stack tecnológico (MEAN, MERN, XAMP, LAMP, WAMP, etc)

Unidad 2. Patrones Arquitectónicos

Número de semanas: 6

- ☒ Catálogo de Patrones Arquitectónicos POSA (Tuberías y Filtros, Blackboard, Brokers, MVC, Procesamiento por lotes, Cliente / Servidor (2, 3y N Capas), Layers, Microkernel, SOA, Arquitectura Hexagonal, Microservicios, WebHooks, BFF, Circuit Breaker, SOFEA)
- ☒ Estilos Arquitectónicos -
- ☒ Principios de Diseño de Software (SOLID, DRY, KISS, DIE, YAGNY) orientados a la integración de componentes
- ☒ Arquitecturas para aplicaciones modernas (JAMStack, WebAssembly, Websockets, gRPC - ProtolBuffer, Microfrontends, WebRTC, Persistencia Políglota). Ciclo DevOPs
- ☒ Anti patrones de Arquitectura de Software

Unidad 3. Desarrollo de SW basado en Arquitectura

Número de semanas: 2

- ☒ Desarrollo basado en Componentes
- ☒ Proceso de Desarrollo orientado a la Arquitectura
- ☒ Middleware
- ☒ Frameworks (Introducción)
- ☒ Implementación de Capa de Persistencia, Presentación, Controlador, de Lógica del Negocio, Capa Modelo de Dominio y Capa de Servicios
- ☒ Mapeo Objeto Relacional ORM

Unidad 4. Los servicios Web en la Arquitectura

Número de semanas: 2

- ☒ Esquemas para el uso de servicios web
- ☒ Arquitecturas Orientadas a Servicios (SOA)
- ☒ Estándares y protocolos para los servicios web (SOAP , WSDL, UDDI y Modelo RestFul, HATEOAS, HAL, GraphQL)
- ☒ Web APIs
- ☒ Implementación de servicios web
- ☒ Arquitecturas Orientadas a la Nube – Cloud Native
- ☒ Arquitectura Orientada a Microservicios (DDD, CQRS, Event Sourcing, API Gateway)
- ☒ Introducción a Serverless Computing (FaaS)

Unidad 5. Aspectos de Seguridad en Arquitecturas de Software

Número de Semanas: 1

- ☒ Estudio de Principios de Arquitectura de software Segura.
- ☒ Importancia de la seguridad en las APIs

6. METODOLOGÍA (SUGERIDA)

Estrategias didácticas:

Aprendizaje Basado en Problemas (ABP), Aprendizaje invertido, Taller

Metodología(s) utilizada(s):

El curso virtual Arquitectura de Software, ofrecido a los estudiantes de quinto semestre del programa de Ingeniería de Sistemas, se compone de 5 unidades temáticas, en las cuales el estudiante realiza un trabajo de apropiación conceptual y crítica frente a la arquitectura de software y, posteriormente, realiza un trabajo colaborativo para la aplicación de metodologías y herramientas en el desarrollo de aplicaciones de software. La implementación de conceptos se realiza de dos formas: a través de la realización de seis talleres a nivel tutorial y en la construcción de un proyecto del curso el cual se divide por módulos. Se espera que el estudiante alcance un aprendizaje autónomo, que posteriormente sea complementado por el trabajo cooperativo con su grupo de trabajo. El producto final del curso es el diseño e implementación del proyecto del curso, elaborado en equipos de trabajo, el cual es transversal a todas las unidades temáticas. El proyecto se divide en dos entregas parciales y una presentación final, así:

-Entrega 1: construcción de las vistas arquitectónicas del sistema y una prueba de concepto funcional básica del módulo escogido.

-Entrega 2: integración de componentes a través de un framework evaluado previamente por los estudiantes, así como el diseño y uso de un servicio web básico usado en el módulo.

Además, se tienen otras actividades evaluativas (talleres, lecturas, foros de discusión, exposiciones y cuestionarios) apoyadas en recursos de aprendizaje disponibles en el aula y la plataforma Moodle, como videos, documentos, animaciones gráficas, explicativos, guías de estudio y autoevaluaciones, todo ello encaminado a afianzar el conocimiento teórico-práctico de los estudiantes, tanto a nivel individual como colaborativo. Desde el comienzo del curso se tienen disponibles los siguientes medios de comunicación: el foro de novedades, en el que se informa de manera permanente sobre la programación y diferentes eventos; el foro de preguntas al tutor, mediante el cual se pueden resolver inquietudes respecto al desarrollo del curso, y la cafetería, que es un foro social donde los estudiantes pueden interactuar con sus compañeros.

Medios y recursos didácticos:

Los recursos didácticos empleados en el curso se basa principalmente en los textos de lectura, tanto artículos científicos como libros, en formato digital para su acceso más disponible e inmediato; también se hace uso de dispositivos multimedia como computadores y televisores para presentaciones visuales de diapositivas, videos, grabaciones o lecturas grupales de textos

Formas de interacción en los ambientes de aprendizaje y de acompañamiento del trabajo independiente del estudiante:

Esta asignatura será guiada en los espacios presenciales por la orientación conceptual del docente hacia el trabajo de preparación previo que los estudiantes han realizado sobre la temática particular a tratar en la sesión gracias a la adopción del Aula Invertida; por lo tanto un tema será abordado en cuatro momentos:

Preparación, consulta e investigación conceptual por cuenta del estudiante y su pequeño grupo de trabajo.

Tratamiento conceptual del tema en sesión del gran grupo junto con el docente.

Aplicación de talleres individuales y cooperativos a nivel tutorial.

Desarrollo de actividades de refuerzo en sesiones autónomas.

En el curso se conformarán equipos de trabajo los cuales desempeñarán el papel de empresas productoras de software, por lo tanto en cada una de ellas deberá existir un líder (que podrá ser representado por cada uno de los integrantes durante las diferentes revisiones del proyecto)

En lo posible se continuará con los proyectos que se vienen realizando en las materias del área de Ingeniería de Software, es decir, que se tomará como insumo para la construcción, pruebas y funcionamiento total del caso de estudio, las etapas de planeación, análisis y diseño realizados en materias anteriores.

Es importante destacar que el 50% de las actividades evaluativas del curso serán enfocadas en las actividades desarrolladas en los correspondientes equipos de trabajo en la Fábrica Escuela de Software y según el avance de los sprints definidos.

Estrategias de internacionalización del currículo que se desarrollan para cumplir con las intencionalidades formativas del microcurrículo:

La internacionalización del currículo en el curso de Arquitectura de Software se centra en preparar a los estudiantes para enfrentar desafíos globales en el campo de la ingeniería de software y promover una perspectiva internacional en su formación. A continuación se presentan algunas estrategias específicas que se pueden implementar para cumplir con las intencionalidades formativas del microcurrículo:

Inclusión de Estudios de Caso Internacionales:

Integrar estudios de caso de proyectos de arquitectura de software que hayan sido implementados internacionalmente. Esto permite a los estudiantes analizar y comprender soluciones arquitectónicas aplicadas en diferentes contextos culturales y geográficos.

Colaboración con Instituciones Extranjeras:

Fomentar la colaboración con instituciones extranjeras como iSAQB International Software Architecture Qualification Board (Acuerdo firmado con la UdeA), a través de proyectos conjuntos o certificaciones internacionales a los estudiantes. La participación en equipos internacionales expone a los estudiantes a diversas perspectivas y prácticas en el diseño de arquitecturas de software.

Idioma

Tener en el curso material en inglés, incluyendo exámenes y actividades académicas permitiendo que los estudiantes se familiaricen con la terminología técnica. Esto facilita la comunicación y colaboración en contextos internacionales.

Análisis de Tendencias Tecnológicas Globales:

Integrar el análisis de tendencias tecnológicas globales en el contenido del curso. Esto incluye la exploración de arquitecturas emergentes, metodologías de desarrollo y prácticas de ingeniería de software que están siendo adoptadas a nivel internacional.

Entorno Multicultural en el Aula:

Fomentar un entorno multicultural en el aula, ya sea a través de la diversidad de estudiantes a través de la estrategia COIL o mediante la participación de profesionales invitados de diferentes partes del mundo. Las discusiones enriquecedoras pueden surgir al abordar problemas desde diversas perspectivas culturales.

Estrategias para abordar o visibilizar la diversidad desde la perspectiva de género, el enfoque diferencial o el enfoque intercultural:

Incorporar estrategias para abordar y visibilizar la diversidad desde la perspectiva de género, el enfoque diferencial y el enfoque intercultural en el curso de Arquitectura de Software es fundamental para promover la equidad y la inclusión. Entre las principales estrategias planteadas para el curso están:

Promoción de Modelos a Seguir Diversos:

Destacar y promover modelos a seguir y expertos en arquitectura de software que representen la diversidad de género y cultural. Se invita a expertos o se incluyen lecturas y presentaciones de profesionales diversos en la industria de la tecnología.

Uso de Lenguaje Inclusivo:

Adoptar un lenguaje inclusivo en la documentación del curso, presentaciones y comunicaciones. Esto implica el uso de términos y pronombres que sean respetuosos y no excluyentes.

Proyectos Colaborativos e Inclusivos:

Fomentar proyectos colaborativos que promuevan la inclusión y diversidad. Establecer equipos de trabajo diversos donde se valoren las diferentes habilidades y perspectivas. Esto refleja la realidad de los entornos laborales en la industria de la tecnología.

Análisis de Impacto Diferencial:

Integrar el análisis de impacto diferencial en el diseño de arquitecturas de software. Enseñar a los estudiantes a considerar cómo sus decisiones pueden afectar de manera diferente a diversos grupos de usuarios en términos de género, cultural o diferencias en habilidades.

Diálogos Abiertos y Foros de Discusión:

Fomentar diálogos abiertos y foros de discusión sobre temas de enfoque diferencial e interculturalidad en el contexto de la arquitectura de software. Crear un espacio seguro donde los estudiantes puedan compartir experiencias y aprender unos de otros.

Evaluación Inclusiva:

Desarrollar estrategias de evaluación que reconozcan y valoren la diversidad de enfoques y soluciones. Evitar la imposición de un único estándar y reconocer la riqueza de perspectivas

7. EVALUACIÓN (SUGERIDA)

Concepción de evaluación, modalidades y estrategias a través de las cuales se va a orientar:

La evaluación se vincula directamente al desarrollo y alcance del proyecto en la fábrica escuela de software identificando dos entregables principales: un documento con las vistas arquitectónicas en el que se aplican los conocimientos que se van trabajando en el curso, y una implementación del BackEnd del proyecto de acuerdo a lo definido en la planificación.

Se hace uso de estrategias de Hetero-evaluación y evaluación entre pares, primando la primera a través de rúbricas de evaluación previamente socializadas

Procesos y resultados de aprendizaje del Programa Académico que se abordan en el curso (según el Acuerdo Académico 583 de 2021 y la Política Institucional):

El curso Arquitectura de Software contribuye, en alguna medida, a los siguientes Resultados de Aprendizaje del programa:

(RA1) Capacidad para identificar, formular y resolver problemas complejos de ingeniería aplicando principios de ingeniería, ciencia y matemáticas.

RA2) Capacidad para aplicar el diseño de ingeniería para producir soluciones que satisfagan necesidades específicas, considerando la salud pública, la seguridad y el bienestar, así como factores globales, culturales, sociales, ambientales y económicos.

(RA3) Capacidad para comunicarse eficazmente con diversas audiencias.

(RA4) Capacidad para reconocer las responsabilidades éticas y profesionales en situaciones de ingeniería y tomar decisiones informadas, considerando el impacto de las soluciones de ingeniería en contextos globales, económicos, ambientales y sociales.

(RA5) Capacidad para trabajar eficazmente en un equipo cuyos miembros, en conjunto, proporcionan liderazgo, crean un entorno colaborativo e inclusivo, establecen metas, planifican tareas y cumplen objetivos (En este curso se realiza una medición de tipo formativo para evaluar este resultado de aprendizaje).

(RA7) Capacidad para adquirir y aplicar nuevos conocimientos según sea necesario, utilizando estrategias de aprendizaje apropiadas

Momentos de evaluación del curso y sus respectivos porcentajes

Momento de evaluación	Porcentaje
Laboratorios, trabajos de clase y exposiciones	30 %
Parcial Teórico y Parcial Práctico	20 %
Documento de Arquitectura CodeF@ctory (Módulo seleccionado)	30 %

Seguimientos sobre actividades del proyecto (3 sprints)	20 %
---	------

8. BIBLIOGRAFÍA Y OTRAS FUENTES		
Cultura o zona geográfica	Bibliografía	Palabras clave
Estados Unidos	BASS, Len, Clements, Paul. KAZMAN, Rick. Software Architecture in Practice. Third Edition 2015. Ed Addison Wesley	Software Architecture, Architectural Styles, Models, Components
Estados Unidos	SHAW Y GALAN. SOFTWARE ARCHITECTURE : PERSPECTIVES ON AN EMERGING DISCIPLINE. Prentice Hall- 2010	Software Architecture, Architectural Styles, Models, Components
Estados Unidos	Erl, Thomas. "SOA: Principles of Service Design". ISBN: 0132344823, Prentice Hall, 2007.	Services, SOA, WebService, SOAP, REST, Orquestación, Coreografía
Estados Unidos	.YOURDON, Edward. Análisis Estructurado Moderno. Ed Prentice Hall. 1993	Modelamiento, UML, Componentes, Diagramas, Casos de Uso
Estados Unidos	SZYPERSKI, C. Component Software: Beyond Object- Oriented Programming. Ed. Addison Wesley. 1997	Componentes
Estados Unidos	GAMMA, Erich, HELM Richard, JOHNSON Ralph y VEISSIDES John. Design Patterns. Ed. Addison Wesley. 1995,	Patrones de Diseño, POSA, Buenas prácticas.
Estados Unidos	BOOCH, Grady, RUMBAUGH, James, JACOBSON, Ivar. El Lenguaje Unificado de Modelado, UML. Ed. Addisson Wesley. 1999	UML, Modelamiento de software
Estados Unidos	FOWLER, Martin y SCOOTT, Kendall. UML Gota a Gota. Ed. Addison Wesley Longman. 1999.	UML, Modelamiento de software
Estados Unidos	Rozanski, N., & Woods, E. Título: Software Systems Architecture: Working with Stakeholders Using Viewpoints and Perspectives Editorial: Addison-Wesley Año de publicación: 2011 (2da edición)	Software Architecture, Architectural Styles, Models, Components
Estados Unidos	Clements, P., Bachmann, F., Bass, L., Garlan, D., Ivers, J., Little, R., Merson, P., Nord, R., & Stafford, J. Título: Documenting Software Architectures:	Software Architecture, Architectural Styles, Models,

	Views and Beyond Editorial: Addison-Wesley Año de publicación: 2010 (2da edición)	Components
Estados Unidos	Fowler, M. Título: Patterns of Enterprise Application Architecture Editorial: Addison-Wesley Año de publicación: 2002	Software Architecture, Architectural Styles, Models, Components
Estados Unidos	Martin, R. C. Título: Clean Architecture: A Craftsman's Guide to Software Structure and Design Editorial: Prentice Hall Año de publicación: 2017	Software Architecture, Architectural Styles, Models, Components
Estados Unidos	Evans, E. Título: Domain-Driven Design: Tackling Complexity in the Heart of Software Editorial: Addison-Wesley Año de publicación: 2003	Microservicios, SOA, CQRS, Event Sourcing, DDD, Api Gateway
Estados Unidos	Sam Newman Título: Building Microservices: Designing Fine-Grained Systems Editorial: O'Reilly Media Año de publicación: 2015	Microservicios, SOA, CQRS, Event Sourcing, DDD, Api Gateway
Estados Unidos	Chris Richardson Título: Microservices Patterns: With Examples in Java Editorial: Manning Publications Año de publicación: 2018	Microservicios, SOA, CQRS, Event Sourcing, DDD, Api Gateway
Estados Unidos	Sam Newman Título: Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith Editorial: O'Reilly Media Año de publicación: 2019	Microservicios, SOA, CQRS, Event Sourcing, DDD, Api Gateway
Estados Unidos	Gigi Sayfan Título: Mastering Kubernetes: Unlocking the Power of the Cluster Orchestration Platform Editorial: O'Reilly Media Año de publicación: 2017	Docker, Contenedores, Broker, Mesh

9. COMUNIDAD ACADÉMICA QUE PARTICIPÓ EN LA ELABORACIÓN DEL MICROCURRÍCULO

Nombres y Apellidos	Unidad académica	Formación académica	% de participación
Diego José Luis Botia Valderrama	Ingeniería de Sistemas	Doctor en Ingeniería Electrónica	100

Aprobado por Comité de Carrera con acta 771 del 11 de Septiembre de 2025
Aprobado en acta de Consejo de Facultad 2507 del 24 de Septiembre de 2025